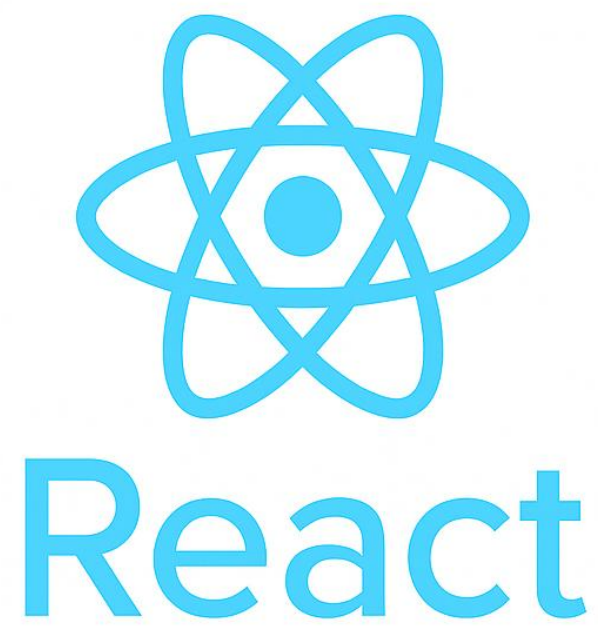




Contents

1.Introduction.....	3
2. Library vs Framework.....	4
3. React vs React Native.....	4
4. Virtual DOM in React	5
4. Single Page Application	6
5. create a React app using Vite	6
6. React Components	10
7. Styles in React.....	12
8. Props in React.....	13
9.Conditional Rendering in React.....	17
10. Dynamic Content loading Using Map.....	19
11. Key Attribute for Components in React.....	23
12. Handling Missing Data in React.....	27
13. Sort and filter function	27
14. Click Event Handling in React.....	29
15 . Hooks In React	30
16. useState Hook	32
17. useEffect Hook	36
18. Async() Function.....	40
19. Fetch Api Data.....	43
20.Host the data in JSON.....	46
21. Handling Error in React	50
22. setTimeout() Function in React	51
23. Custom Hook in React	54
24.Onchange Event Handler	56
25. Router.....	60
26.Cleanup Function	63
27.Updater Function	66
28.UseContext.....	69
29.UseRef.....	71
30.UseNavigate.....	73
31. Dynamic Path.....	79
32. React Component Lifecycle	84

1.Introduction

React JS (commonly called **React**) is an **open-source JavaScript library** developed by Facebook for building **user interfaces**, especially for **single-page applications (SPAs)**. It focuses on creating reusable UI components and efficiently updating the user interface when data changes.

Key Features of React JS

1. **Component-Based Architecture**
 - UI is broken down into small, reusable pieces called components.
 - Each component manages its own state and logic.
2. **Virtual DOM**
 - React uses a virtual representation of the DOM to optimize updates.
 - Instead of updating the entire DOM, React updates only the parts that change, improving performance.
3. **Declarative UI**
 - You describe what the UI should look like, and React handles the rendering and updates.
4. **Unidirectional Data Flow**
 - Data flows in one direction (from parent to child), making the app predictable and easier to debug.
5. **JSX (JavaScript XML)**
 - A syntax extension that allows you to write HTML-like code inside JavaScript.
6. **Ecosystem & Community**
 - Large ecosystem with libraries like **React Router** (for routing) and **Redux** (for state management).

Why Use React?

- High performance due to Virtual DOM.
- Easy to build dynamic and interactive UIs.
- Strong community support and wide adoption.
- Works well with other libraries and frameworks.

2. Library vs Framework

✓ Library

- A library is a **collection of reusable code** (functions, classes, utilities) that you can call when needed.
 - **You control the flow** of the application. You decide when and how to use the library.
 - Example:
 - **React** is a library. You use its functions (like `useState`, `useEffect`) to build UI, but you decide the overall structure.
 - **Analogy:** A library is like a toolbox—you pick the tools you need.
-

✓ Framework

- A framework is a **complete structure** for building applications.
 - **It controls the flow:** You follow its rules and conventions, and it calls your code at specific points.
 - Example:
 - **Angular** is a framework. It dictates how components, routing, and data binding should work.
 - **Analogy:** A framework is like a house blueprint—you build according to its design.
-

3. React vs React Native

✓ React (React.js)

- **What it is:**
A **JavaScript library** for building **web user interfaces**.
 - **Platform:**
Runs in the **browser** (web applications).
 - **Language:**
Uses **JavaScript** and **JSX**.
 - **Purpose:**
Creates **interactive, dynamic web pages**.
 - **Rendering:**
Uses the **Virtual DOM** to update HTML efficiently.
 - **Example Use Case:**
Building a website like **Udemy** or **Facebook**.
-

✓ React Native

- **What it is:**
A **framework** for building **mobile applications** using React concepts.
 - **Platform:**
Runs on **iOS and Android** (mobile apps).
 - **Language:**
Uses **JavaScript + React**, but renders **native components** (not HTML).
 - **Purpose:**
Creates **native mobile apps** with a single codebase.
 - **Rendering:**
Uses **native UI components** (like `View`, `Text`) instead of HTML tags.
 - **Example Use Case:**
Building a mobile app like **Instagram** or **Uber**.
-

4. Virtual DOM in React

✓ What is the Virtual DOM?

- The **DOM (Document Object Model)** is the structure of your web page in the browser.
 - Updating the real DOM frequently is **slow** because it involves re-rendering elements and recalculating styles.
 - **Virtual DOM** is a **lightweight copy of the real DOM** that React keeps in memory.
-

✓ How It Works

1. When your app's **state changes**, React:
 - Updates the **Virtual DOM** first (not the real DOM).
 2. React then **compares** the new Virtual DOM with the previous one using a process called **diffing**.
 3. It finds the **minimal changes** needed and updates only those parts in the **real DOM**.
 - This is called **reconciliation**.
-

✓ Why is it Important?

- **Performance:** Instead of updating the entire page, React updates only what changed.
- **Efficiency:** Reduces costly operations on the real DOM.
- **Smooth UI:** Makes apps faster and more responsive.

4. Single Page Application

A **Single Page Application (SPA)** is a type of web application that **loads a single HTML page** and dynamically updates the content as the user interacts with the app, **without reloading the entire page**.

✓ Key Characteristics of SPA

1. **Single Initial Load**
 - The app loads once, and then only data is fetched from the server as needed.
 2. **Dynamic Content Updates**
 - Uses **JavaScript** (often with frameworks like React, Angular, Vue) to update parts of the page without a full refresh.
 3. **Client-Side Routing**
 - Navigation between "pages" happens on the client side using libraries like **React Router**, so the URL changes but the page doesn't reload.
 4. **Better User Experience**
 - Faster and smoother because it avoids full page reloads.
-

✓ How It Works

- When you click a link in an SPA:
 - Instead of requesting a new HTML page from the server, the app updates the view using JavaScript.
 - Data is fetched via **APIs (AJAX or Fetch)**.
-

5. create a React app using Vite

0) Prerequisites

1. **Install Node.js** (LTS recommended, e.g., v18+ or v20+).
 - Verify:

```
node -v
```

```
npm -v
```

What these do:

- `node -v` shows your Node version (runtime for JS outside the browser).
- `npm -v` shows your npm version (package manager).

- `npm i -g <tool>` installs a tool globally so you can run it from any folder.

1) Create a Vite project (React)

Using npm (recommended and simplest)

```
npm create vite@latest
```

Explanation:

- `npm create` is like `npx` for project scaffolding.
- `vite@latest` fetches the latest Vite initializer.
- This command will prompt you for:
 - **Project name** (folder to create)
 - **Framework** → select React
 - **Variant** → select JavaScript or TypeScript

You can also shortcut with arguments:

JavaScript variant

```
npm create vite@latest my-react-app -- --template react
```

TypeScript variant

```
npm create vite@latest my-react-app -- --template react-ts
```

Explanation of flags:

- `my-react-app` → the **directory name** to create.
- `--` → separates npm's arguments from Vite's arguments.
- `--template react` → tells Vite to scaffold a React app (JS).
- `--template react-ts` → React + TypeScript template.

2) Move into the project and install dependencies

After scaffolding:

```
cd my-react-app
```

```
npm install
```

Explanation:

- `cd my-react-app` changes the current working directory into your new project.

- `npm install` reads your `package.json` and installs dependencies into `node_modules` (e.g., `vite`, `react`, `react-dom`).
-

3) Start the development server

`npm run dev`

Explanation:

- Runs the `dev` script defined in `package.json`, which typically is:

```
"scripts": {  
  
  "dev": "vite",  
  
  "build": "vite build",  
  
  "preview": "vite preview"  
}
```

- `vite` starts a local dev server (default port **5173**) with **hot module replacement (HMR)** so changes appear instantly without full reload.

Output (typical):

```
VITE vX.X.X ready in XX ms
```

```
→ Local:   http://localhost:5173/
```

```
→ Network: use --host to expose
```

Open the **Local** URL in your browser to see your React app.

4) Build for production (optional now, essential later)

`npm run build`

Explanation:

- Runs `vite build`, which:
 - Bundles your code with **Rollup**
 - Generates optimized assets in the `dist/` folder (minified JS/CSS)
 - Handles tree-shaking, code splitting, etc.

5) Preview production build (quick local check)

npm run preview

Explanation:

- Serves the static files from `dist/` with Vite's preview server, to verify production build behavior locally.

6) Project structure—what did the template create?

Typical files (React + JS):

```
my-react-app/
├─ node_modules/      # Installed dependencies
├─ public/             # Static assets (served as-is)
├─ src/
│  ├─ assets/          # Example images or styles
│  ├─ App.jsx          # Main component
│  ├─ main.jsx         # Entry point (mounts React to DOM)
│  └─ index.css        # Global styles
├─ .gitignore          # Files to ignore in Git
├─ index.html          # Single HTML entry (SPA root)
├─ package.json        # Scripts and dependencies
├─ vite.config.js      # Vite configuration
└─ README.md           # Basic instructions
```

Key pieces:

- `index.html` contains `<div id="root"></div>` where React mounts.
- `main.jsx` imports React and attaches `<App />` to `#root`.
- `App.jsx` is the main React component.
- `vite.config.js` can be customized (aliases, plugins, server options).

7) Common adjustments you'll likely make

Enable network access (test on mobile)

npm run dev -- --host

Explanation:

- `--host` lets your dev server listen on your machine's IP so other devices on your LAN can access it.
- You'll see a **Network** URL printed (e.g., `http://192.168.1.5:5173`).

Install a UI library (example)

`npm install react-router-dom`

Explanation:

- Adds client-side routing capability for SPA navigation.

8) Typical commands—summary with why/when

- `npm create vite@latest` → Scaffold a new Vite project (interactive prompts).
- `npm create vite@latest my-react-app -- --template react` → Scaffold React JS directly with name.
- `cd my-react-app` → Enter the project.
- `npm install` → Install dependencies listed in `package.json`.
- `npm run dev` → Start dev server (with HMR).
- `npm run build` → Build optimized production files in `dist/`.
- `npm run preview` → Serve the production build locally to test.

6. React Components

✅ What is a React Component?

A component is a **reusable piece of UI** in React. It can be:

- **Functional Component** (modern, recommended)
- **Class Component** (older style, still used in some projects)

To Create A Functional Component using short command in VS Code.

rfce

✅ 1. Functional Component (Recommended)

Here's the simplest example:

```
// App.jsx
```

```
function Greeting() {  
  return <h1>Hello, Welcome to React!</h1>;  
}
```

```
export default Greeting;
```

Explanation:

- `function Greeting()` → Defines a component named `Greeting`.
- `return <h1>...</h1>` → JSX (HTML-like syntax) returned by the component.
- `export default Greeting;` → Makes it available for import elsewhere.

Usage in main file:

```
import Greeting from './Greeting';
```

```
function App() {  
  return (  
    <div>  
      <Greeting />  
    </div>  
  );  
}
```

```
export default App;
```

✅ 2. Class Component (Older Style)

```
import React, { Component } from 'react';  
class Greeting extends Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}  
  
export default Greeting;
```

7. Styles in React

1. Inline Styles

You can use the `style` attribute with a JavaScript object:

```
function App() {  
  return (  
    <div style={{ color: 'blue', fontSize: '20px' }}>  
      Hello, React!  
    </div>  
  );  
}
```

- **Pros:** Simple, scoped to the component.
- **Cons:** No pseudo-classes (`:hover`), no media queries.

2. CSS Stylesheets

Create a `.css` file and import it:

```
/* App.css */  
  
.container {  
  color: blue;  
  font-size: 20px;  
}  
  
import './App.css';
```

```
function App() {  
  return <div className="container">Hello, React!</div>;  
}
```

- **Pros:** Familiar, supports all CSS features.
- **Cons:** Global scope unless you use CSS Modules.

3. CSS Modules

Scoped CSS by default:

```
/* App.module.css */  
.container {  
  color: blue;  
  font-size: 20px;  
}  
  
import styles from './App.module.css';  
  
function App() {  
  return <div className={styles.container}>Hello, React!</div>;  
}
```

- **Pros:** Avoids class name collisions.
- **Cons:** Requires build setup.

4. Tailwind CSS or Bootstrap

Utility-first CSS framework:

```
function App() {  
  return <div className="text-blue-500 text-xl">Hello, React!</div>;  
}
```

- **Pros:** Fast, responsive utilities.
- **Cons:** Learning curve for class names.

5. Global Stylesheet which is Index.css

8. Props in React

✓ 1. Using the `props` keyword

This is the classic way where you receive `props` as a parameter and access its properties:

```
function Greeting(props) {
```

```
    return <h1>Hello, {props.name}!</h1>;  
  }
```

// Usage

```
<Greeting name="Karthikeyan" />
```

Here:

- `props` is an object containing all the properties passed to the component.
- You access each property using `props.propertyName`.

✅ 2. Using Destructuring

A cleaner way is to destructure props directly:

```
function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

// Usage

```
<Greeting name="Karthikeyan" />
```

This avoids repeatedly writing `props.` and makes code more readable.

✅ Default Props

Default props provide fallback values if a prop is not passed.

For Functional Components:

```
function Greeting({ name = "Guest" }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

// Usage

```
<Greeting /> // Output: Hello, Guest!
```

Using `defaultProps` (older approach):

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

```
Greeting.defaultProps = {  
  name: "Guest"  
};
```

1) Basics: Passing and Using Props

```
```jsx  
// Parent
function CourseCard() {
 return <PriceTag amount={499} currency="INR" />;
}
// Child
function PriceTag({ amount, currency }) {
 return {currency}{amount};
}
```
```

- * Props are passed like HTML attributes.
- * In the child, you typically **destructure** props for readability.

2) Prop Types (JS) or TypeScript (recommended)

With PropTypes (JavaScript)

```
```jsx  
import PropTypes from 'prop-types';
function CourseTitle({ title, featured }) {
 return <h2 className={featured ? "featured" : ""}>{title}</h2>;
}
CourseTitle.propTypes = {
 title: PropTypes.string.isRequired,
 featured: PropTypes.bool,
};
```
```

3) Avoiding Prop Drilling

****Prop drilling**** = passing props through multiple layers only to be used deep down. Prefer:

* ****Context**** for global/shared data (e.g., current user, theme)

* ****State management**** (Redux, Zustand, Jotai) for complex flows

```
` `` `jsx
const UserContext = React.createContext(null);

function App() {
  const [user] = React.useState({ name: "Karthikeyan" });
  return (
    <UserContext.Provider value={user}>
      <Layout />
    </UserContext.Provider>
  );
}

function Header() {
  const user = React.useContext(UserContext);
  return <span>Welcome, {user?.name}</span>;
}
` `` `
```

Passing **methods (functions) via props** in React is how a **child component communicates back to its parent**—e.g., when a button is clicked in a child, the parent can update state. This is often called “**passing callbacks**”.

1) Basic: Pass a Function from Parent to Child

```
// Parent.jsx
import React from "react";
import Child from "./Child";

export default function Parent() {
  function handleGreet() {
    alert("Hello from Parent!");
  }

  return <Child onGreet={handleGreet} />;
}
```



```

}

// Child.jsx

export default function Child({ onGreet }) {

  return <button onClick={onGreet}>Greet</button>;

}

```

- `Parent` defines `handleGreet`.
- It passes the function as a prop: `onGreet={handleGreet}`.
- `Child` calls it (e.g., on click).

Naming convention: use `onSomething` for props that are callbacks (like DOM events).

2) Pass Arguments (Data) Back to Parent

```

// Parent.jsx

export default function Parent() {

  function onSelectCourse(courseId) {

    console.log("Selected:", courseId);

  }

  return <CourseCard id="c101" onSelect={onSelectCourse} />;

}

```

```

// CourseCard.jsx

export default function CourseCard({ id, onSelect }) {

  return <button onClick={() => onSelect(id)}>Select Course</button>;

}

```

- The child wraps the callback in an arrow function to **pass parameters**.
-

9. Conditional Rendering in React

✓ 1. Using `if` Statements

```

function Greeting({ isLoggedIn }) {

  if (isLoggedIn) {

```

```
    return <h1>Welcome back!</h1>;  
  }  
  return <h1>Please log in.</h1>;  
}
```

✅ 2. Using Ternary Operator

Best for inline conditions:

```
function Greeting({ isLoggedIn }) {  
  return (  
    <h1>{isLoggedIn ? "Welcome back!" : "Please log in."}</h1>  
  );  
}
```

✅ 3. Using Logical AND (&&)

Render something only if the condition is true:

```
function Notification({ hasMessage }) {  
  return (  
    <div>  
      {hasMessage && <p>You have new messages!</p>}  
    </div>  
  );  
}
```

✅ 4. Using `switch` or Multiple Conditions

For multiple cases:

```
function Status({ status }) {  
  switch (status) {  
    case "loading":  
      return <p>Loading...</p>;  
  }  
}
```

```
case "success":
  return <p>Data loaded!</p>;
case "error":
  return <p>Error occurred!</p>;
default:
  return null;
}
}
```

✓ 5. Conditional Rendering with Components

You can render different components based on conditions:

```
function App({ isLoggedIn }) {
  return isLoggedIn ? <Dashboard /> : <Login />;
}
```

✓ 6. Using `null` to Hide Elements

If you return `null`, React renders nothing:

```
function Banner({ show }) {
  if (!show) return null;
  return <div className="banner">Welcome!</div>;
}
```

10. Dynamic Content loading Using Map

1) Basic: Render a List with ``map``

```

` `` `jsx
function CourseList({ courses }) {
  return (
    <ul>
      {courses.map(course => (
        <li key={course.id}>
          {course.title} — ₹{course.price}
        </li>
      ))}
    </ul>
  );
}

// Usage
const courses = [
  { id: "c1", title: "React Basics", price: 499 },
  { id: "c2", title: "Node.js API", price: 699 },
];
<CourseList courses={courses} />;
` `` `

```

****Key points****

- * Always set a **stable `key`** (use a unique id from your data, not the array index).
- * ``map`` transforms your data array into an array of React elements.

2) Map with Conditional Rendering

Show extra info based on item properties:

```

` `` `jsx
function CourseGrid({ courses }) {
  return (
    <div className="grid">
      {courses.map(course => (
        <div key={course.id} className="card">
          <h3>{course.title}</h3>
          <p>₹{course.price}</p>
          {course.featured && <span className="badge">Featured</span>}
          {!course.published && <em>Coming soon</em>}}
        </div>
      ))}
    </div>
  );
}
` `` `

```

You can also **filter** before mapping****** for readability:

```

` `` `jsx
{courses
  .filter(c => c.published)
  .map(c => <CourseCard key={c.id} course={c} />)}
` `` `

```

3) Map with Loading, Error & Empty States

When loading dynamic content from an API, combine **loading states** with ``map``:

```
```.jsx
```

```
import { useEffect, useState } from "react";
```

```
function CoursesPage() {
```

```
 const [courses, setCourses] = useState([]);
```

```
 const [loading, setLoading] = useState(true);
```

```
 const [error, setError] = useState("");
```

```
 useEffect(() => {
```

```
 let active = true;
```

```
 setLoading(true);
```

```
 fetch("/api/courses")
```

```
 .then(r => r.json())
```

```
 .then(data => { if (active) setCourses(data); })
```

```
 .catch(err => setError(err.message))
```

```
 .finally(() => { if (active) setLoading(false); });
```

```
 return () => { active = false; };
```

```
 }, []);
```

```
 if (loading) {
```

```
 return (
```

```
 <div className="grid">
```

```
 {Array.from({ length: 8 }).map((_, i) => (
```

```
 <div key={i} className="skeleton-card" />
```

```
))}
```

```
 </div>
```

```
);
```

```
 }
```

```
if (error) return <p className="error">Failed to load: {error}</p>;
```

```
if (courses.length === 0) return <p>No courses found.</p>;
```

```
return (
```

```
 <div className="grid">
```

```
 {courses.map(c => <CourseCard key={c.id} course={c} />)}
```

```
 </div>
```

```
);
```

```
}
```

```
...`
```

### ## Best Practices for `map` in React

- \* **Keys must be stable and unique**: Prefer `item.id`. Avoid `index` (causes reconciliation bugs on reorder/insert/delete).

- \* **Pure components & memoization**: If a list is large, wrap rows with `React.memo` to avoid unnecessary re-renders.

- \* **Avoid heavy computations inside `map`**: Precompute outside or memoize with `useMemo`.

- \* **Empty states**: Always handle `[]` gracefully (show a helpful message).

- \* **Skeletons over spinners**: Use skeletons for lists to reduce layout shift.

- \* **Accessibility**: When mapping interactive items, ensure focus order and ARIA attributes are correct.

---

## 11. Key Attribute for Components in React

🧠 What is a `key`?

A `key` is a **stable, unique identifier** for an element within its list. React uses keys during **reconciliation** to decide:

- \* Which items are **added**, **removed**, or **reordered**

- \* Whether a child component should be **updated** or **remounted** (thus preserving or resetting state)

> Keys are **not passed** to your component as props; they're used internally by React.

\*\*\*

## ✅ When to use `key`

Use `key` whenever you **render lists** with `.map()` (or any array iteration):

```
```jsx
function CourseList({ courses }) {
  return (
    <ul>
      {courses.map(course => (
        <li key={course.id}>{course.title}</li> // ✅ stable, unique key
      ))}
    </ul>
  );
}
```

✅ Good key choices

* A **stable unique ID** from your data (`id`, `slug`, etc.)

* A **composite key** if no single ID exists:

```
```jsx
key={` ${userId}:${courseId} `}
...

```

## ## ❌ Avoid these key anti-patterns

### 1) **Array index as key**

```
```jsx
// ❌ Using index
items.map((item, index) => <Row key={index} {...item} />);
...
***
```

Why it's bad (when items can be inserted, deleted, or reordered):

* React may **reuse the wrong DOM** and component state for different items

* Causes subtle bugs (e.g., wrong input values, wrong focus, animation glitches)

When index is acceptable:

- * The list is **static** (no insert/remove/reorder)
- * Items never change position and are **purely presentational**

2) **Non-unique or unstable keys**

- * Randomly generated each render (`Math.random()`, new UUIDs): causes **full remounts** every time
- * Values that can **change** across renders (e.g., array position)

🎯 Why keys matter: preserving component state

```jsx

```
function TodoList({ todos }) {
 return todos.map(todo => <TodoItem key={todo.id} todo={todo} />);
}

function TodoItem({ todo }) {
 const [checked, setChecked] = useState(false);

 // With stable keys, toggling items preserves the right checkbox state
 return (
 <label>
 <input
 type="checkbox"
 checked={checked}
 onChange={() => setChecked(c => !c)}
 />
 {todo.text}
 </label>
);
}
```

## 🛠️ Practical Guidelines

1. **Prefer IDs from the backend**. If you don't have one, create a **stable client-side ID** when data arrives (and store it alongside).
2. **Never use the array index** when the list can **change order**, **insert**, or **delete**.

3. Keys are **local to their list**; ensure uniqueness **within the siblings** being iterated.
4. For **grouped content**, put keys on the **closest repeated element** (not the inner arbitrary child).
5. In **forms or interactive lists**, using correct keys prevents **state leakage** between rows.
6. For performance, **stable keys** help React avoid unnecessary DOM operations.

\*\*\*

## ## 🧩 Examples: Correct vs Incorrect

**Incorrect (index key; reordering breaks state):**

```
```jsx
{students.map((s, i) => (
  <StudentRow key={i} student={s} />
))}
```
```

**Correct (stable unique key):**

```
```jsx
{students.map(s => (
  <StudentRow key={s.id} student={s} />
))}
```
```

**Composite key when no ID:**

```
```jsx
{enrollments.map(e => (
  <EnrollmentRow key={` ${e.studentEmail}-${e.courseCode}`} enrollment={e} />
))}
```
```

## 12. Handling Missing Data in React

### 1) Defensive Rendering in JSX

Optional chaining & nullish coalescing

Use these to avoid runtime errors and provide sensible fallbacks.

```
function CourseCard({ course }) {
 return (
 <article className="card">
 <h3>{course?.title ?? "Untitled course"}</h3>
 <p>By {course?.instructor ?? "Unknown instructor"}</p>
 <p>₹{course?.price ?? "—"}</p>
 <p>{course?.description?.slice(0, 120) ?? "No description available."}</p>
 </article>
);
}
```

- `?.` safely accesses nested properties.
- `??` falls back **only** when the left side is `null` or `undefined` (unlike `||`, which treats `0`, `false`, `""` as falsy).

### 2) Loading, Error, and Empty States (LE&E)

Build consistent, reusable states.

```
function ErrorMessage({ message = "Something went wrong.", onRetry }) {
 return (
 <div role="alert" className="error">
 {message}
 {onRetry && <button onClick={onRetry}>Retry</button>}
 </div>
);
}
```

```
function EmptyState({ text = "No data available." }) {
 return <p className="empty">{text}</p>;
}
```

---

## 13. Sort and filter function

### ✅ Filter Example (Plain React)

You have an array and want to show only items that match a condition:

```
const courses = ["React", "Node.js", "JavaScript", "TypeScript"];

const filteredCourses = courses.filter(course => course.includes("React"));
```

```
function App() {
 return (

 {filteredCourses.map((course, index) => (
 <li key={index}>{course}
))}

);
}
```

---

### ✓ Sort Example

Sort alphabetically:

```
const courses = ["React", "Node.js", "JavaScript", "TypeScript"];
const sortedCourses = [...courses].sort(); // Ascending order

function App() {
 return (

 {sortedCourses.map((course, index) => (
 <li key={index}>{course}
))}

);
}
```

### ✓ Sort Numbers in Ascending Order

```
const numbers = [40, 10, 30, 20];
const sortedNumbers = [...numbers].sort((a, b) => a - b);

function App() {
 return (

 {sortedNumbers.map((num, index) => (
```

```
 <li key={index}>{num}
)}

);
}
```

---

#### ✔ Sort Numbers in Descending Order

```
const numbers = [40, 10, 30, 20];
const sortedNumbers = [...numbers].sort((a, b) => b - a);
function App() {
 return (

 {sortedNumbers.map((num, index) => (
 <li key={index}>{num}
))}

);
}
```

---

To view output in vs code inside simple browser show.

---

## 14. Click Event Handling in React

Click event handling in React is very simple. You use the `onClick` attribute on an element and pass a function that should run when the element is clicked.

---

#### ✔ Basic Example

```
function App() {
 function handleClick() {
 alert("Button clicked!");
 }
}
```

```
return (
 <button onClick={handleClick}>Click Me</button>
);
}
```

#### Explanation:

- `onClick` is a React event handler.
- `handleClick` is the function that runs when the button is clicked.

#### ✅ Passing Arguments

```
function App() {
 function greet(name) {
 alert(`Hello, ${name}`);
 }

 return (
 <button onClick={() => greet("Karthikeyan")}>Greet</button>
);
}
```

#### Key Points for Beginners:

- Use `onClick` in JSX (camelCase, not `onclick` like HTML).
  - Pass a **function**, not the result of a function (don't write `onClick={handleClick() }`).
  - You can use inline arrow functions for simple cases.
- 

## 15 . Hooks In React

#### ✅ What Are Hooks?

Hooks are **special functions introduced in React 16.8** that let you use **state, lifecycle methods**, and other React features inside **functional components**.

Before Hooks:

- Functional components were **stateless** (just UI).
- Class components handled **state** and **lifecycle**.

Hooks changed this by allowing **functional components to do everything classes can do**—but in a simpler, reusable way.

---

## ✅ Why Hooks Were Introduced

- **Avoid complexity of classes** (binding `this`, lifecycle confusion).
- **Reuse logic easily** (custom hooks instead of HOCs or render props).
- **Better code organization** (logic grouped by feature, not lifecycle method).

---

## ✅ Rules of Hooks

1. **Only call Hooks at the top level** (not inside loops, conditions, or nested functions).
2. **Only call Hooks inside React components or custom hooks** (not in regular JS functions).

React enforces these rules using the **Rules of Hooks** ESLint plugin.

---

## ✅ Core Hooks

### 1. `useState` – State Management

---

### 2. `useEffect` – Side Effects

Runs code **after render** (like lifecycle methods `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`).

---

### 3. `useContext` – Global State

Access data from React Context without prop drilling.

---

### 4. `useReducer` – Complex State

Alternative to `useState` for complex logic (similar to Redux).

---

### 5. `useRef` – Persistent Value

Stores a value that **doesn't cause re-render** (e.g., DOM reference or mutable variable).

---

### 6. Custom Hooks

---

## ✅ Why Hooks Are Powerful

- No need for classes.
- Easier to share logic (custom hooks).
- Cleaner code (group related logic together).
- Mode\*\* and **Suspense**.

---

## 🔥 Summary:

Hooks = **functions that let you use React features in functional components.**

- `useState` → state
- `useEffect` → side effects
- `useContext` → global data
- `useReducer` → complex state
- `useRef` → persistent values

---

# 16. useState Hook

## ✅ What is `useState`?

`useState` is a **React Hook** that lets you **add state** (data that changes over time) to a **functional component**.

Before Hooks, only **class components** could have state. With `useState`, functional components can store and update data easily.

---

## ✅ Syntax

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

- `stateVariable` → current value of the state.
- `setStateFunction` → function to update the state.
- `initialValue` → starting value of the state.

---

## ✅ Example

```
import React, { useState } from "react";

function Counter() {
```



```

const [count, setCount] = useState(0); // initial value = 0

return (
 <div>
 <p>Count: {count}</p>
 <button onClick={() => setCount(count + 1)}>Increase</button>
 </div>
);
}

```

### What happens here?

- `useState(0)` creates a state variable `count` with initial value 0.
- `setCount` updates the value.
- When you click the button, `count` increases and React **re-renders** the component with the new value.

---

#### ✅ Example with Object

```

function UserInfo() {
 const [user, setUser] = useState({ name: "Karthikeyan", age: 22 });
 return (
 <div>
 <p>Name: {user.name}</p>
 <button onClick={() => setUser({ ...user, age: user.age + 1 })}>
 Increase Age
 </button>
 </div>
);
}

```

---

#### ✅ Why use `useState`?

- To make your UI **dynamic**.
- To store data that changes (like form inputs, counters, toggles).
- To replace class component state with a simpler approach.

---

### ✅ Basic Setup

```
import React, { useState } from "react";

export default function Numbers() {

 const [nums, setNums] = useState([10, 20, 30]); // initial array

 return (

 <div>

 {nums.map((n, i) => (

 <li key={i}>{n}

))}

 </div>

);

}
```

- `nums` is your array state.
- `setNums` updates the array and **re-renders** the UI.

⚠️ **Important:** Never mutate state directly (e.g., `nums.push(40)`). Always create a **new array** when updating.

---

### + Add Items

Append to the end

```
setNums(prev => [...prev, 40]);
```

Add to the beginning

```
setNums(prev => [5, ...prev]);
```

Add multiple

```
setNums(prev => [...prev, 100, 200]);
```

---

### — Remove Items

By index

```
setNums(prev => prev.filter((_, index) => index !== 1)); // remove item at index 1
```

By value

```
setNums(prev => prev.filter(n => n !== 20)); // remove all 20s
```

---

### Update/Replace Items

Update by index

```
setNums(prev => prev.map((n, i) => (i === 1 ? n + 10 : n))); // add 10 to item at index 1
```

Update by condition (e.g., all even numbers +1)

```
setNums(prev => prev.map(n => (n % 2 === 0 ? n + 1 : n)));
```

---

### Functional Updates (Best Practice)

When the next state depends on the previous state, use the **functional form**:

```
setNums(prev => [...prev, Math.floor(Math.random() * 100)]);
```

This avoids bugs when multiple updates happen quickly.

---

### Sort & Filter (Derived Views)

Sorting mutates arrays, so **clone before sorting**.

```
const sortedAsc = [...nums].sort((a, b) => a - b);
```

```
const sortedDesc = [...nums].sort((a, b) => b - a);
```

```
const onlyEven = nums.filter(n => n % 2 === 0);
```

If you need to **store** sorted results in state:

```
setNums(prev => [...prev].sort((a, b) => a - b));
```

---

### Reset Array

```
setNums([]); // clear all
```

```
setNums([10, 20, 30]); // reset to initial
```

---

### Common Mistakes (and Fixes)

-  `nums.push(40); setNums(nums);` \ **Why bad?** Mutates the same array; React may not detect change correctly.
-  `setNums(prev => [...prev, 40]);` \ **Why good?** Creates a **new array**; React re-renders reliably.
-  Updating multiple times without functional updates:

```
setNums([...nums, 1]);
```

```
setNums([...nums, 2]); // both use old nums
```

-  Use functional form:

```
setNums(prev => [...prev, 1]);
```

```
setNums(prev => [...prev, 2]);
```

---

## 17. useEffect Hook

What is `useEffect`?

`useEffect` is a React hook that lets you run **side effects** in function components after React has rendered to the DOM.

**Side effects** = anything that touches the “outside world” or needs to run *after* render: data fetching, subscriptions, timers, logging, updating document title, syncing state to `localStorage`, etc.

```
import { useEffect, useState } from "react";
```

```
function Example() {
```

```
 const [count, setCount] = useState(0);
```

```
 // Runs after render when count changes
```

```
 import { useEffect, useState } from "react";
```

```
function Example() {
```

```
 const [count, setCount] = useState(0);
```

```
 // Runs after render when count changes
```

```
 useEffect(() => {
```

```
 document.title = `Clicked ${count} times`;
 }, [count]);
}
```

---

When should I use it?

Use `useEffect` for **post-render work**:

1. **Fetching data** from APIs.
2. **Subscribing** to events (WebSocket, DOM events) and **cleaning up** on unmount.
3. **Starting/stopping timers** or intervals.
4. **Syncing external state** (e.g., `localStorage`) with React state.
5. **Reacting to prop/state changes** to perform a side effect (e.g., refetch when `id` changes).
6. **Integrations** with non-React libraries (e.g., map libraries, charts) that need DOM.

---

Why do we use it?

- Ensures effects run at the **right time**: after React updates the UI.
  - Keeps side-effect logic **declarative**: “run this when these values change.”
  - Provides a **cleanup** mechanism to avoid memory leaks and stale subscriptions.
  - Plays well with React’s lifecycle and concurrent rendering.
- 

What should I know before using it?

1) Dependency array matters

- `useEffect(fn) →` runs on **every render**.
- `useEffect(fn, []) →` runs **once** after first render (mount).
- `useEffect(fn, [a, b]) →` runs when **any dependency changes**.

React compares dependencies by **reference** (not deep equality). If you recreate objects/functions every render, the effect will re-run.

2) Cleanup function

Return a function from the effect to **clean up** subscriptions, timers, or pending tasks.

```
useEffect(() => {
 const id = setInterval(() => { /* ... */ }, 1000);
```

```
return () => clearInterval(id); // runs on unmount or before re-running
}, []);
```

### 3) Avoid infinite loops

Do **not** update state unconditionally inside an effect without a stable condition/dependency—it will re-render and re-run the effect forever.

### 4) Stale closures & missing deps

- Always include **all values used inside the effect** in the dependencies.
- If that causes too many reruns, consider memoizing (`useCallback`, `useMemo`) or move logic to event handlers.

### 5) React 18 Strict Mode (dev only)

In development with Strict Mode, React will **run effects twice** to surface bugs (mount → cleanup → mount). This doesn't happen in production, but you must write effects that are **idempotent** (safe to run/cleanup repeatedly).

### 6) `useEffect` VS `useLayoutEffect`

- `useEffect` runs **after paint** (non-blocking). Most effects should use this.
- `useLayoutEffect` runs **before paint** (blocking)—use only for **layout reads/writes** where flicker must be avoided.

### 7) Server Components / SSR

`useEffect` **does not run on the server**. Effects only run in the browser.

---

## Pitfalls & best practices

-  **Include all dependencies** used in the effect body. Use ESLint's `react-hooks/exhaustive-deps` rule.
-  **Cleanup on unmount** (events, timers, subscriptions, observers).
-  **Keep effects focused**—split one big effect into multiple smaller ones for clarity.
-  **Make effects idempotent** (safe to run more than once), especially in dev with Strict Mode.
-  **Don't fetch in render**; only inside effects or event handlers.
-  **Don't put React state setters in dependency arrays** unless they're custom or memoized; built-in setters are stable.
-  **Avoid object/array literals** as dependencies—extract them or memoize.
-  **Guard async effects** against race conditions with a `cancelled` flag or `AbortController`.

---

## Quick mental model

- **Render** (pure) → **Commit to DOM** → **useEffect runs** → (optional) **Cleanup before next run/unmount**.
- Declare **when** to run using the **dependency array**.

---

Example: Fetch user data when `userId` changes (with cleanup)

```
import { useEffect, useState } from "react";

function UserCard({ userId }: { userId: string }) {
 const [user, setUser] = useState<any>(null);
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState<string | null>(null);

 useEffect(() => {
 // Create an abort controller to cancel fetch if userId changes or component unmounts
 const controller = new AbortController();

 async function loadUser() {
 setLoading(true);
 setError(null);

 try {
 const res = await fetch(`/api/users/${userId}`, { signal: controller.signal });
 if (!res.ok) throw new Error(`HTTP ${res.status}`);
 const data = await res.json();
 setUser(data); // safe: runs after render
 } catch (e: any) {
 // Ignore abort errors; report real errors
 if (e.name !== "AbortError") setError(e.message);
 } finally {
 setLoading(false);
 }
 }

 loadUser();

 // Cleanup: abort the request if this effect re-runs or component unmounts
 return () => controller.abort();
 }, [userId]); // <-- effect runs whenever userId changes

 if (loading) return <p>Loading...</p>;
 if (error) return <p>Error: {error}</p>;
 if (!user) return <p>No user</p>;
 return (
 <div>
 <h3>{user.name}</h3>
 </div>
);
}
```

```
 <p>Email: {user.email}</p>
 </div>
);
}
```

- **Render happens first** → React paints the UI.
  - **Effect runs** → starts the fetch using the current `userId`.
  - If `userId` changes:
    - React calls the **cleanup** (abort the in-flight request).
    - Then re-runs the effect with the new `userId`.
  - **State updates** (`setUser`, `setLoading`, `setError`) trigger re-renders, and the UI shows loading/error/data accordingly.
- 

## 18. Async() Function

### ✓ What is an `async` function?

An `async` function in JavaScript is a special function that **always returns a Promise** and allows you to use the `await` keyword inside it.

- `async` makes your function asynchronous.
- `await` pauses the function execution until the Promise resolves (or rejects).

This gives you a way to write asynchronous code that **looks like synchronous code**, making it easier to read and maintain.

---

### ✓ Why do we use `async`?

JavaScript is **single-threaded**, so long-running tasks (like API calls, file reads, database queries) would block the UI if done synchronously.

We use `async` functions to:

- **Handle asynchronous operations** without blocking the main thread.
  - **Avoid callback hell** and make code cleaner than using `.then()` chains.
  - **Write readable code** that looks sequential but runs asynchronously.
- 

### ✓ How does it work?

- When you call an `async` function, it **returns a Promise immediately**.
- Inside the function, you can use `await` to wait for another Promise to resolve before continuing.



- If the function throws an error, it becomes a **rejected Promise**.

---

✅ Example: Fetching data with `async/await`

```
async function fetchUser(userId) {
 try {
 // Wait for the fetch to complete
 const response = await fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);

 // Wait for the JSON parsing
 const user = await response.json();

 console.log("User:", user);
 return user; // returns a Promise that resolves to user
 } catch (error) {
 console.error("Error fetching user:", error);
 throw error; // rejected Promise
 }
}

// Call the async function
fetchUser(1).then(user => console.log("Fetched:", user));
```

---

✅ Why is this better than `.then()`?

Without `async/await`:

```
fetch(https://jsonplaceholder.typicode.com/users/1)
 .then(response => response.json())
 .then(user => console.log(user))
 .catch(error => console.error(error));
```

With `async/await`, the flow looks **linear**, easier to read, and easier to handle errors with `try/catch`.

---

✅ Key things to know before using `async`:

1. **Always use `await` inside an `async` function** (not in global scope unless using top-level `await` in modern environments).
2. **Error handling:** Use `try/catch` for errors or `.catch()` on the returned Promise.
3. **Parallel vs sequential:**
  - `await` runs sequentially by default.
  - For parallel tasks, use `Promise.all()`.

Example:

```
async function fetchTwoUsers() {
 const [user1, user2] = await Promise.all([
 fetchUser(1),
 fetchUser(2)
]);
 console.log(user1, user2);
}
```

---

✅ In React, why do we often use `async` inside `useEffect`?

Because data fetching is asynchronous. But **you cannot make `useEffect` itself `async`**—instead, define an `async` function inside it and call it:

```
useEffect(() => {
 async function loadData() {
 const res = await fetch("/api/data");
 const data = await res.json();
 setState(data);
 }
 loadData();
}, []);
```

---

✅ What is a Promise?

- Think of a Promise as a **placeholder for a value** that you don't have yet but will get in the future.
  - It's used to handle **asynchronous tasks** like API calls, file reads, timers, etc.
- 

✅ States of a Promise

A Promise can be in one of three states:

1. **Pending** – The operation is still running.
2. **Fulfilled** – The operation completed successfully, and the Promise has a value.

3. **Rejected** – The operation failed, and the Promise has a reason (error).

---

✅ Why do we use Promises?

Before Promises, we used **callbacks**, which often led to **callback hell** (nested callbacks that are hard to read and maintain).\ Promises make asynchronous code **cleaner and easier to chain**.

---

✅ Example: Basic Promise

```
const myPromise = new Promise((resolve, reject) => {
 const success = true; // simulate condition

 if (success) {
 resolve("Operation successful!");
 } else {
 reject("Operation failed!");
 }
});

myPromise
 .then(result => console.log(result)) // runs if resolved
 .catch(error => console.error(error)) // runs if rejected
 .finally(() => console.log("Done")); // runs always
```

---

✅ Real-world Example: Fetch API returns a Promise

```
fetch("https://jsonplaceholder.typicode.com/users/1")
 .then(response => response.json()) // response.json() also returns a Promise
 .then(data => console.log("User:", data))
 .catch(error => console.error("Error:", error));
```

---

## 19. Fetch Api Data

✅ Basic Fetch Example (JavaScript)

```

fetch("https://jsonplaceholder.typicode.com/users/1")
 .then(response => {
 if (!response.ok) {
 throw new Error("Network response was not ok");
 }
 return response.json(); // Convert response to JSON
 })
 .then(data => {
 console.log("User Data:", data);
 })
 .catch(error => {
 console.error("Fetch error:", error);
 });

```

How it works

1. `fetch(url)` returns a **Promise**.
2. `.then(response => response.json())` parses the JSON body.
3. `.catch(error => ...)` handles network or parsing errors.

---

### ✅ Using `async/await` (Cleaner way)

```

async function getUser() {
 try {
 const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
 if (!response.ok) {
 throw new Error("Network response was not ok");
 }
 const data = await response.json();
 console.log("User Data:", data);
 } catch (error) {
 console.error("Error fetching data:", error);
 }
}

```

getUser();

---

✅ In React (with `useEffect`)

```
import { useEffect, useState } from "react";

function UserComponent() {
 const [user, setUser] = useState(null);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);
 useEffect(() => {
 async function fetchUser() {
 try {
 const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
 if (!response.ok) throw new Error("Failed to fetch");
 const data = await response.json();
 setUser(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
 }
 fetchUser();
 }, []);
 if (loading) return <p>Loading...</p>;
 if (error) return <p>Error: {error}</p>;
 return <pre>{JSON.stringify(user, null, 2)}</pre>;
}
```

For Axios

Axios.get()-get data

Axios.put()-update data

Axios.post()-create data

Axios.delete()-delete data

---

## 20.Host the data in JSON

### ✓ Why use json-server?

- No backend coding required.
- You can **simulate GET, POST, PUT, DELETE** easily.
- Perfect for testing API calls in React before building a real backend.

---

### ✓ Steps in Detail:

#### 1. Create a JSON file

Make a file named `db.json` in your project root:

```
{
 "courses": [
 {
 "id": 1,
 "title": "React Basics",
 "author": "MK",
 "image": "/images/react-basics.jpg"
 },
 {
 "id": 2,
 "title": "Node Essentials",
 "author": "MK",
 "image": "/images/node-essentials.jpg"
 }
]
}
```

Here:

- `courses` is your resource name.
  - Each object has an `image` path pointing to your React `public/images` folder.
- 

## 2. Put images in React `public` folder

Inside your React project:

```
public/
 images/
 react-basics.jpg
 node-essentials.jpg
```

This makes them available at:

```
http://localhost:3000/images/react-basics.jpg
```

---

## 3. Install and run `json-server`

Run this command in your project root:

```
npx json-server --watch db.json --port 4000
```

- `--watch db.json` → watches your JSON file for changes.
- `--port 4000` → API will run on `http://localhost:4000`.

Now your API endpoints:

```
GET http://localhost:4000/courses
```

---

## 4. Fetch data in React

Example component:

```
import { useEffect, useState } from "react";
```

```
function Courses() {
 const [courses, setCourses] = useState([]);
 const [loading, setLoading] = useState(true);
```

```

const [error, setError] = useState(null);
useEffect(() => {
 async function fetchCourses() {
 try {
 const res = await fetch("http://localhost:4000/courses");
 if (!res.ok) throw new Error("Failed to fetch");
 const data = await res.json();
 setCourses(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
 }
 fetchCourses();
}, []);
if (loading) return <p>Loading...</p>;
if (error) return <p>Error: {error}</p>;
return (

 {courses.map(course => (
 <li key={course.id}>

 <p>{course.title} — {course.author}</p>

))}

);
}
export default Courses;

```



---

Great—you're on the right track. Here's how that command works, the correct folder setup, and how to fetch both **API data** and **static images** served by `json-server`.

---

```
npx json-server --watch db.json --port 4000 --static ./assets
```

- `--watch db.json`: starts a fake REST API using `db.json` and watches for changes.
- `--port 4000`: serves the API at `http://localhost:4000`.
- `--static ./assets`: serves **static files** (images, CSS, JS) from the `assets` folder at the **same origin** (e.g., `http://localhost:4000/images/...`).

**Note:** You typed `./asserts`—that's likely a typo. Use `./assets`.

---

#### ✅ Recommended folder structure

```
your-project/
├ db.json
├ assets/
│ └ images/
│ ├── react-basics.jpg
│ └ node-essentials.jpg
└ src/ (your React app)
```

#### `db.json` example:

```
{
 "courses": [
 {
 "id": 1,
 "title": "React Basics",
 "author": "MK",
 "image": "/images/react-basics.jpg"
 },
 {
 "id": 2,
 "title": "Node Essentials",
 "author": "MK",
```

```
"image": "/images/node-essentials.jpg"
}
]
}
```

- Because you're using `--static ./assets`, requests to `/images/...` map to `assets/images/...`
- Full image URLs will be: `http://localhost:4000/images/react-basics.jpg`.

---

### ✓ Start the server

Run:

```
npx json-server --watch db.json --port 4000 --static ./assets
```

You'll get:

- API: `http://localhost:4000/courses`
- Images: `http://localhost:4000/images/react-basics.jpg`

---

## 21. Handling Error in React

### ✓ 1. Why Error Handling Matters in React

React apps often involve:

- **Component rendering** (UI logic)
- **Async operations** (API calls, data fetching)
- **State management** (Redux, Context) Errors can occur in any of these layers, so handling them properly prevents crashes and improves UX.

---

### ✓ 2. Types of Errors in React

- **Render-time errors:** When a component throws during rendering.
  - **Event handler errors:** Errors inside `onClick`, `onChange`, etc.
  - **Async errors:** API failures, promises rejected.
  - **Global errors:** Uncaught exceptions in the app.
-

### ✅ 3. Handling Errors in Event Handlers

React doesn't automatically catch these. Use `try...catch`:

```
function handleClick() {
 try {
 // risky code
 } catch (error) {
 console.error(error);
 }
}
```

---

### ✅ 4. Handling Async Errors

For API calls:

```
async function fetchData() {
 try {
 const response = await fetch('/api/data');
 if (!response.ok) throw new Error('Network error');
 const data = await response.json();
 } catch (error) {
 console.error(error);
 }
}
```

---

## 22. SetTimeout() Function in React

### 1) Basic usage in a component

You can use `setTimeout` anywhere (it's a browser API), but in React you typically use it inside effects or event handlers.

In an event handler (simple):

```
function Button() {
```

```

const handleClick = () => {
 setTimeout(() => {
 alert('Runs after 1s');
 }, 1000);
};

return <button onClick={handleClick}>Click me</button>;
}

```

---

## 2) In `useEffect` with proper cleanup

When you start a timeout during render/effect, **always clear it** on unmount or when the effect reruns. This prevents memory leaks and actions on unmounted components.

```

import { useEffect } from 'react';

function DelayedMessage({ text, delay = 1000 }) {
 useEffect(() => {
 const id = setTimeout(() => {
 console.log(text);
 }, delay);

 return () => clearTimeout(id); // cleanup on unmount or dep change
 }, [text, delay]);

 return <div>Message will log after {delay} ms</div>;
}

```

---

## 3) Avoiding stale closures

By default, the function passed to `setTimeout` captures the state/props at the time it was created. If you need the **latest values**, either:

- Include them in the `useEffect` dependency array so the timeout is refreshed, or

- Store the latest value in a `ref` and read from it inside the timeout.

Using a ref to always read the latest state:

```
import { useEffect, useRef, useState } from 'react';

function Example() {
 const [count, setCount] = useState(0);
 const latestCountRef = useRef(count);

 useEffect(() => {
 latestCountRef.current = count;
 }, [count]);

 useEffect(() => {
 const id = setTimeout(() => {
 // Always reads the latest value from the ref
 console.log('Latest count:', latestCountRef.current);
 }, 1000);
 return () => clearTimeout(id);
 }, []); // set up once

 return (
 <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
);
}
```

## 23. Custom Hook in React

What is a Custom Hook?

A **custom hook** is just a function that:

- Starts with `use...`
- Uses React hooks inside (like `useState`, `useEffect`, `useRef`)
- Encapsulates reusable logic (state, effects, event wiring), **not** UI

You can share behavior across components without duplicating code or creating HOCs/render props.

---

1) `useFetch` — data fetching with loading/error and abort

**Use case:** Simple GET requests (no heavy state libs).

```
import { useEffect, useState } from 'react';

/**
 * Fetches JSON from a URL. Handles loading, error, and abort on unmount or URL change.
 */
export function useFetch(url, options) {
 const [data, setData] = useState(null);
 const [loading, setLoading] = useState(Boolean(url));
 const [error, setError] = useState(null);

 useEffect(() => {
 if (!url) return;

 const controller = new AbortController();
 const { signal } = controller;

 let mounted = true;
 setLoading(true);
 setError(null);

 (async () => {
 try {
 const res = await fetch(url, { ...options, signal });
 if (!res.ok) throw new Error(`HTTP ${res.status}`);
 const json = await res.json();
 if (mounted) setData(json);
 } catch (err) {
 if (err.name === 'AbortError') return;
 if (mounted) setError(err);
 } finally {

```

```

 if (mounted) setLoading(false);
 }
})());

return () => {
 mounted = false;
 controller.abort();
};
}, [url, JSON.stringify(options)]); // stringify to avoid re-runs unless options change
meaningfully

return { data, loading, error };
}

```

### Usage:

```

function Users() {

 const { data, loading, error } = useFetch('/api/users');

 if (loading) return <p>Loading...</p>;
 if (error) return <p>Error: {error.message}</p>;

 return (

 {data.map(u => <li key={u.id}>{u.name})}

);
}

```

---

## 2) useToggle — tiny state helper

**Use case:** Modal open/close, checkbox toggles.

```
import { useCallback, useState } from 'react';
```

```
export function useToggle(initial = false) {
 const [on, setOn] = useState(initial);

```

```

const toggle = useCallback(() => setOn(o => !o), []);
const setTrue = useCallback(() => setOn(true), []);
const setFalse = useCallback(() => setOn(false), []);

return { on, toggle, setTrue, setFalse };
}

```

### Usage:

```

function ModalExample() {
 const { on, toggle } = useToggle();
 return (
 <>
 <button onClick={toggle}>Toggle Modal</button>
 {on && <div className="modal">Hello</div>}
 </>
);
}

```

---

## 24. onChange Event Handler

What is `onChange` in React?

`onChange` is a prop you attach to form elements (like `<input>`, `<textarea>`, `<select>`) to run your code whenever the user **changes the value**. In **controlled components**, you use it to keep the UI in sync with React state.

React's `onChange` is slightly different from the browser's `change` event:

- For text inputs, it fires on **every keystroke** (like `input` event in the browser).
- For checkboxes/radios, it fires when the **checked** state changes.
- For `<select>`, it fires when the selected option changes.

---

The standard controlled input pattern

```
import { useState } from 'react';
```

```
function NameField() {
```



```

const [name, setName] = useState("");

const handleChange = (e) => {
 setName(e.target.value); // keep state in sync with input
};

return (
 <div>
 <input value={name} onChange={handleChange} placeholder="Your name" />
 <p>Typed: {name}</p>
 </div>
);
}

```

### Key points:

- The input's `value` comes from state.
- `onChange` updates that state.
- This is called a **controlled component**.

---

### Different input types

#### Text / number

```
<input type="text" value={value} onChange={e => setValue(e.target.value)} />
```

// number: convert to a number (or handle empty)

```

<input
 type="number"
 value={age}
 onChange={e => setAge(e.target.value === "" ? "" : Number(e.target.value))}
/>

```

#### Checkbox

```
const [agree, setAgree] = useState(false);
```

```
<input
 type="checkbox"
 checked={agree}
 onChange={e => setAgree(e.target.checked)}
/>
```

Radio

```
const [plan, setPlan] = useState('basic');
```

```
<label>
 <input
 type="radio"
 name="plan"
 value="basic"
 checked={plan === 'basic'}
 onChange={e => setPlan(e.target.value)}
 />
```

Basic

```
</label>
```

```
<label>
 <input
 type="radio"
 name="plan"
 value="pro"
 checked={plan === 'pro'}
 onChange={e => setPlan(e.target.value)}
 />
```

Pro

```
</label>
```

Select

```
const [country, setCountry] = useState('IN');
```

```
<select value={country} onChange={e => setCountry(e.target.value)}>
 <option value="IN">India</option>
 <option value="US">USA</option>
 <option value="DE">Germany</option>
</select>
```

Textarea

```
<textarea value={bio} onChange={e => setBio(e.target.value)} />
```

---

Handling multiple fields succinctly

```
const [form, setForm] = useState({ email: '', password: '' });
```

```
function handleChange(e) {
 const { name, value, type, checked } = e.target;
 setForm(prev => ({
 ...prev,
 [name]: type === 'checkbox' ? checked : value
 }));
}
```

```
<form>
 <input name="email" value={form.email} onChange={handleChange} />
 <input name="password" type="password" value={form.password} onChange={handleChange} />
 <input name="remember" type="checkbox" checked={form.remember || false} />
 onChange={handleChange} />
</form>
```

---

Common pitfalls & best practices

- **Controlled vs uncontrolled:**
    - Controlled: use `value` (or `checked`) + `onChange`.
    - Uncontrolled: use `defaultValue/defaultChecked` and read values via refs on submit.
  - **Performance:**
    - `onChange` fires very often for text inputs. If your handler is heavy, **debounce** or **throttle**.
  - **Synthetic event pooling (older React):**
    - If you use the event asynchronously, read values immediately: `const { value } = e.target;` before awaiting. (Modern React no longer pools events, but this pattern is safe.)
  - **Numbers & booleans:**
    - Convert with `Number(...)` or `JSON.parse(...)` when needed.
    - For checkbox, always use `e.target.checked`.
  - **Accessibility:**
    - Labels (`<label htmlFor="...">`) and proper input types make forms more usable.
- 

## 25. Router

### 1) What is React Router?

**React Router** is the standard library for **client-side routing** in React. It lets you map URLs to components, navigate without full page reloads, and manage **nested layouts**, **data loading**, **form actions**, and **errors** in a predictable way.

Install:

```
npm i react-router-dom
```

---

### 2) Core Building Blocks

`<BrowserRouter>` (simple apps)

Wraps your app to enable routing.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
```

```
export default function App() {
```

```
 return (
```

```

<BrowserRouter>
 <Routes>
 <Route path="/" element={<Home/>} />
 <Route path="/courses" element={<Courses/>} />
 </Routes>
</BrowserRouter>
);
}

```

Data Router (recommended for data loading)

Use **createBrowserRouter + RouterProvider** for loaders/actions.

```

import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const router = createBrowserRouter([
 { path: '/', element: <Home/> },
]);
export default () => <RouterProvider router={router} />;

```

### 3) Route Definitions

JSX routes

```

<Routes>
 <Route path="/" element={<Home/>} />
 <Route path="/courses" element={<Courses/>} />
</Routes>

```

Object routes (data router)

```

const router = createBrowserRouter([
 { path: '/', element: <Home/> },
 { path: '/courses', element: <Courses/> }
]);

```

### 4) Navigation

Declarative: `<Link>` & `<NavLink>`

```

import { Link, NavLink } from 'react-router-dom';

```

```
<NavLink to="/courses" className={({isActive}) => isActive ? 'active' : ''}>
```

```
 Courses
```

```
</NavLink>
```

```
<Link to="/dashboard">Dashboard</Link>
```

Programmatic: `useNavigate`

```
import { useNavigate } from 'react-router-dom';
```

```
function LoginButton() {
```

```
 const nav = useNavigate();
```

```
 return <button onClick={() => nav('/dashboard')}>Go to Dashboard</button>;
```

```
}
```

---

## 5) Dynamic Segments & Params

Define `:id` in route; read via `useParams`.

```
// route
```

```
<Route path="/courses/:courseId" element={<CourseDetail/>} />
```

```
// component
```

```
import { useParams } from 'react-router-dom';
```

```
function CourseDetail() {
```

```
 const { courseId } = useParams();
```

```
 return <h2>Course #{courseId}</h2>;
```

```
}
```

## 26.CleanUp Function

💡 What is a Cleanup Function?

A **cleanup function** is the function you **return** from a `useEffect` (or perform in `componentWillUnmount` for class components). React calls it:

- **Before** the effect runs again (when its dependencies change), and
- **When** the component **unmounts**.

Its job is to **undo side effects**: remove event listeners, cancel timers, abort in-flight requests, and stop subscriptions—so your app doesn't leak memory or try to update unmounted components.

```
useEffect(() => {
 // side effect setup

 return () => {
 // cleanup runs here
 };
}, [deps]);
```

---

🚀 Why We Use Cleanup Functions

1. **Prevent memory leaks** (dangling timers, listeners, sockets).
2. **Avoid state updates on unmounted components** (common source of errors).
3. **Stop duplicate side effects** when dependencies change.
4. **Keep behavior correct in React 18 dev Strict Mode**, which intentionally mounts, unmounts, and remounts components to surface side-effect issues.

---

✅ Core Patterns with Examples

### 1) Cleanup for Event Listeners

**Use case:** You attach a `keydown` listener—remove it when the component unmounts or when handler changes.

```
import { useEffect } from 'react';
```

```
function EscToClose({ onClose }) {
 useEffect(() => {
 const handleKey = (e) => {
```

```

 if (e.key === 'Escape') onClose();
 };
 window.addEventListener('keydown', handleKey);

 return () => {
 window.removeEventListener('keydown', handleKey);
 };
}, [onClose]); // re-wire when handler changes
return null;
}

```

**Explanation:**

- We add the listener once.
- When `onClose` changes or on unmount, we remove it to avoid duplicate listeners and leaks.

---

## 2) Cleanup for Timers (`setTimeout` / `setInterval`)

**Use case:** Start a timer; clear it later to avoid running code after unmount.

```

import { useEffect } from 'react';

function Toast({ onExpire, duration = 3000 }) {
 useEffect(() => {
 const id = setTimeout(onExpire, duration);
 return () => clearTimeout(id); // prevents firing after unmount
 }, [onExpire, duration]);

 return <div className="toast">Closes in {duration} ms</div>;
}

```

**Explanation:**

- The timeout schedules `onExpire`.



- Cleanup ensures the timeout is canceled if the component unmounts or the duration/handler changes.

---

### 3) Cleanup for Fetch with AbortController

**Use case:** Cancel in-flight requests when the component unmounts or the URL changes.

```
import { useEffect, useState } from 'react';

function User({ id }) {
 const [user, setUser] = useState(null);
 const [error, setError] = useState(null);

 useEffect(() => {
 const controller = new AbortController();

 (async () => {
 try {
 const res = await fetch(`/api/users/${id}`, { signal: controller.signal });
 if (!res.ok) throw new Error(`HTTP ${res.status}`);
 const json = await res.json();
 setUser(json);
 } catch (err) {
 if (err.name !== 'AbortError') setError(err);
 }
 })();

 return () => controller.abort(); // cancel request
 }, [id]);

 if (error) return <p style={{ color: 'red' }}>{error.message}</p>;
 if (!user) return <p>Loading...</p>;
 return <pre>{JSON.stringify(user, null, 2)}</pre>;
}
```

#### Explanation:

- `AbortController` lets us cancel the request.
- Prevents setting state after unmount if the fetch resolves late.

---

#### Important Notes

- **When does cleanup run?**
  - Right before the effect re-runs due to dependency changes.
  - When the component unmounts.

- **React 18 Strict Mode (development only):**
    - Effects may run twice (mount → cleanup → mount).
    - If your effect performs setup, ensure cleanup correctly undoes it to avoid duplicates.
  - **Avoid side effects directly in render.**
    - Always put side effects (timers, subscriptions, event listeners, network calls) **inside** `useEffect` and clean them up.
- 

## 27.Updater Function

Why we use the Updater function

- **Avoid stale state:** Event handlers and async callbacks may capture old values. The updater function guarantees you get the latest state at the moment of update.
  - **Correct with batching:** Multiple state updates in the same tick are batched; `setX(prev => ...)` ensures each update uses the latest result.
  - **Safe under concurrency** (React 18): When React may re-render or interrupt, the functional update stays correct.
  - **Compose multiple updates:** You can call `setState` several times in a row using the previous state reliably.
- 

The pattern

Function signature

`setState(prevState => nextState)`

- `prevState`: the state value at update time (not the value captured when the handler was defined).
  - Return the new state.
- 

Examples (with explanations)

1) Counter — correct increment with multiple clicks

**Pitfall (incorrect):**

```
const [count, setCount] = useState(0);
```

```
function incrementTwice() {
```

```
 // Both read the same stale count value
```

```
setCount(count + 1);
setCount(count + 1);
}
```

If `count` is 0, after this runs you may still get 1 due to batching (both use 0).

### **Fix (updater function):**

```
function incrementTwice() {
 setCount(prev => prev + 1);
 setCount(prev => prev + 1);
}
```

Now result is 2, because the second update sees the updated `prev`.

---

### 2) Toggle / derived next state

```
const [on, setOn] = useState(false);
```

```
function toggle() {
 setOn(prev => !prev);
}
```

Explanation: Even if multiple toggles are queued, each inverts the *current* value correctly.

---

### 3) Update object state safely

```
const [form, setForm] = useState({ email: '', password: '' });
```

```
function setEmail(email) {
 setForm(prev => ({ ...prev, email }));
}
```

Explanation: We spread the previous object to avoid discarding other fields.

---

### 4) Async handlers and stale closures

**Pitfall:**

```
const [items, setItems] = useState([]);

async function addItemAfterFetch() {
 const data = await fetchItem(); // time passes...
 // items here may be stale
 setItems([...items, data]);
}
```

**Fix:**

```
async function addItemAfterFetch() {
 const data = await fetchItem();
 setItems(prev => [...prev, data]); // always uses the latest list
}
```

---

**5) Class components: `setState` with updater**

```
class Counter extends React.Component {
 state = { count: 0 };

 addThree = () => {
 this.setState(prev => ({ count: prev.count + 1 }));
 this.setState(prev => ({ count: prev.count + 1 }));
 this.setState(prev => ({ count: prev.count + 1 }));
 };

 render() {
 return <button onClick={this.addThree}>{this.state.count}</button>;
 }
}
```

Explanation: Each call uses the latest `prev.count`, so final result is +3 even with batching.

---

#### 6) Queuing multiple dependent updates

```
const [value, setValue] = useState(1);

function complexUpdate() {
 setValue(prev => prev * 2); // 1 → 2
 setValue(prev => prev + 10); // 2 → 12
 setValue(prev => Math.sqrt(prev)); // 12 → ~3.46
}
```

Explanation: Each step sees the **result** of the previous step.

---

#### 7) With arrays: push without stale state

```
const [todos, setTodos] = useState([]);

function addTodo(todo) {
 setTodos(prev => [...prev, todo]);
}
```

Explanation: Spreads previous array and appends; safe under concurrency.

---

#### 8) When you don't need it

If the next state **does not** depend on previous state, you can pass the value directly:

```
setTheme('dark'); // independent of the old theme
```

---

## 28.UseContext

### What is useContext?

`useContext` is a **React Hook** that lets you **consume values from a Context** without writing `<Context.Consumer>` manually. It's part of React's **Context API**, which is used for **state sharing across components** without prop drilling.

---

Why do we use it?

- Avoid **prop drilling** (passing props through multiple levels).
- Share **global state** (like theme, user info, language, auth) across deeply nested components.
- Works well with **state management** patterns for small/medium apps.

---

### ✓ How Context Works

1. **Create a Context** using `React.createContext()`.
2. **Provide** a value using `<Context.Provider>` at a high level.
3. **Consume** the value in any child using `useContext(MyContext)`.

---

### ✓ Example: Theme Switcher (Step-by-Step)

#### 1) Create Context

```
import React, { createContext, useContext, useState } from 'react';
```

```
// Create a context
```

```
const ThemeContext = createContext();
```

---

#### 2) Provide Context in a Parent

```
function App() {
 const [theme, setTheme] = useState('light');

 return (
 <ThemeContext.Provider value={{ theme, setTheme }}>
 <Toolbar />
 </ThemeContext.Provider>
);
}
```

Here:

- `ThemeContext.Provider` wraps the app.

- We pass { `theme`, `setTheme` } so children can read and update theme.

---

✅ Why is this better than prop drilling?

Without context, you'd pass `theme` and `setTheme` through every intermediate component (`App` → `Toolbar` → `ThemeButton`). With `useContext`, any component can access the value directly.

---

## 29.UseRef

✅ What is `useRef`?

`useRef` is a **React Hook** that returns a **mutable object** with a `.current` property. This object:

- **Persists across renders** (does not reset like local variables).
- **Does not cause re-renders** when you change `.current`.
- Can hold **DOM references** or **any mutable value** (like timers, previous state).

---

Why do we use `useRef`?

1. **Access DOM elements** (e.g., focus an input, measure size).
2. **Store values without triggering re-render** (e.g., previous state, timer IDs).
3. **Avoid stale closures** in async callbacks.
4. **Persist data between renders without state overhead.**

---

✅ Syntax

```
const myRef = useRef(initialValue);
```

- `myRef.current` holds the value.
- Changing `myRef.current` does **not** re-render the component.

---

✅ Example 1: Accessing DOM Element (Focus Input)

```
import React, { useRef } from 'react';
```

```
function FocusInput() {
 const inputRef = useRef(null);
```

```

const handleFocus = () => {
 inputRef.current.focus(); // Access DOM node
};

return (
 <div>
 <input ref={inputRef} type="text" placeholder="Type here..." />
 <button onClick={handleFocus}>Focus Input</button>
 </div>
);
}

```

#### Explanation

- `useRef(null)` creates a ref object.
- `ref={inputRef}` attaches it to the `<input>`.
- `inputRef.current` points to the actual DOM element.
- Clicking the button calls `.focus()` on that element.

---

#### ✅ Example 2: Storing Previous State Without Re-render

```
import React, { useEffect, useRef, useState } from 'react';
```

```

function PreviousValueDemo() {
 const [count, setCount] = useState(0);
 const prevCountRef = useRef(count);

 useEffect(() => {
 prevCountRef.current = count; // update after render
 }, [count]);

 return (
 <div>
 <p>Current: {count}</p>

```



```

 <p>Previous: {prevCountRef.current}</p>

 <button onClick={() => setCount(c => c + 1)}>Increment</button>

 </div>

);
}

```

#### Explanation

- `prevCountRef` stores the previous `count` value.
- Unlike state, updating `.current` does **not** trigger a re-render.
- Useful for comparing current vs previous values.

---

#### ✓ Key Points

- `useRef` **does not trigger re-render** when updated.
  - Great for **DOM access, mutable values**, and **avoiding stale closures**.
  - If you need UI updates, use `useState` instead.
- 

## 30.UseNavigate

#### ✓ What is `useNavigate`?

`useNavigate` is a hook from **react-router-dom** that lets you **navigate programmatically** (i.e., in JavaScript) rather than using `<Link>` elements. It updates the URL and renders the matching route **without a full page reload**.

```
import { useNavigate } from 'react-router-dom';
```

```
const navigate = useNavigate();
```

```
navigate('/dashboard'); // go to /dashboard
```

Works inside components rendered within a Router (e.g., `<BrowserRouter>` or `RouterProvider`).

---

#### ✓ Why do we use it?

- **Conditional navigation** (after login, after form submit, on success/failure).

- **Navigate with data** (pass state to the next route).
- **Replace history** (redirects that shouldn't be "back"-navigable).
- **Relative navigation** (move within nested routes).
- **Back/forward stack control** (go -1, +1).

---

### Basic Usage

```
import { useNavigate } from 'react-router-dom';
```

```
function GoButton() {
 const navigate = useNavigate();

 return (
 <button onClick={() => navigate('/courses')}>
 Go to Courses
 </button>
);
}
```

---

### Replace vs Push (history behavior)

- **Push** (default): adds a new entry to history.
- **Replace**: replaces the current entry (useful for redirects).

```
navigate('/login', { replace: true }); // user can't go "Back" to previous
```

---

### Relative Navigation (nested routes)

When inside a nested route, you can navigate relative to the current path:

```
// From /courses/:courseId
navigate('lessons/1'); // => /courses/:courseId/lessons/1
navigate('..'); // go up one segment => /courses
navigate('..create'); // => /courses/create
```

## Navigate Back/Forward

`navigate(-1); // equivalent to history.back()`

`navigate(1); // forward`

---

### ✓ Common Real-World Patterns

#### 1) Redirect after login (replace history)

```
function Login() {
 const navigate = useNavigate();

 async function handleSubmit(e) {
 e.preventDefault();
 const ok = await fakeAuth(); // your API call
 if (ok) {
 navigate('/dashboard', { replace: true }); // avoid going back to login
 }
 }

 return (
 <form onSubmit={handleSubmit}>
 { /* fields */ }
 <button type="submit">Login</button>
 </form>
);
}
```

**Why:** Successful login should feel like a redirect, not a page you can go “Back” to.

---

#### 2) Navigate with a `redirectTo` (preserve intent)

If the user hits a protected page, send them to login with a `redirectTo` query, then return them after login.

```
// Guard (component-level example)
import { useLocation, Navigate } from 'react-router-dom';

function RequireAuth({ authenticated, children }) {
 const loc = useLocation();
 if (!authenticated) {
 return <Navigate to={`/login?redirectTo=${encodeURIComponent(loc.pathname)}`}
 replace />;
 }
 return children;
}
```

```
// In Login after success:
const params = new URLSearchParams(location.search);
const redirectTo = params.get('redirectTo') || '/dashboard';
navigate(redirectTo, { replace: true });
```

**Why:** Preserve user's original destination.

---

### 3) Submit → Navigate with state

```
function Enroll({ courseId }) {
 const navigate = useNavigate();

 async function enroll() {
 const res = await enrollApi(courseId);
 if (res.ok) {
 navigate(`/courses/${courseId}`, { state: { justEnrolled: true } });
 }
 }

 return <button onClick={enroll}>Enroll</button>;
}
```

**Why:** Show a contextual message on the target page without polluting the URL.

---

### 4) Relative navigation in layouts

```
function CourseLayout() {
 const navigate = useNavigate();

 return (
 <>
 <button onClick={() => navigate('..')}>Back to Courses</button>
 { /* <Outlet /> renders child routes */ }
 </>
);
}
```

```

 </>
);
}

```

**Why:** Clean navigation within nested route structures.

---

### Tips & Best Practices

- Use `replace: true` for **redirect-style** navigations (login, 404, post-submit).
- Prefer `state` for ephemeral data instead of query strings when it doesn't need to be shareable/bookmarkable.
- In **nested routes**, prefer **relative paths** for maintainable navigation.
- For **data mutations and navigation**, consider **React Router's <Form> + actions** when appropriate; otherwise `useNavigate` is perfect for client-side-only flows.
- Ensure you call `useNavigate` **inside** components (or custom hooks called by components) rendered within a Router.

---

### Minimal, End-to-End Example

```

import { BrowserRouter, Routes, Route, useNavigate, useLocation, Navigate } from 'react-router-dom';
import { useState } from 'react';

```

```

function App() {
 return (
 <BrowserRouter>
 <Routes>
 <Route path="/" element={<Home/>} />
 <Route path="/login" element={<Login/>} />
 <Route
 path="/dashboard"
 element={
 <RequireAuth>
 <Dashboard/>
 </RequireAuth>
 }
 />
 <Route path="*" element={<NotFound/>} />
 </Routes>
 </BrowserRouter>
);
}

```

```

function Home() {
 const navigate = useNavigate();

```

```

return (
 <
 <h1>Home</h1>
 <button onClick={() => navigate('/dashboard')}>Go to Dashboard</button>
 </>
);
}

function RequireAuth({ children }) {
 const [authed] = useState(false); // replace with real auth
 const loc = useLocation();
 if (!authed) {
 return <Navigate to={` /login?redirectTo=${encodeURIComponent(loc.pathname)}`}
replace />;
 }
 return children;
}

function Login() {
 const navigate = useNavigate();
 const loc = useLocation();

 async function handleLogin() {
 // simulate login
 await new Promise(r => setTimeout(r, 400));
 const params = new URLSearchParams(loc.search);
 const redirectTo = params.get('redirectTo') || '/dashboard';
 navigate(redirectTo, { replace: true });
 }

 return (
 <
 <h2>Login</h2>
 <button onClick={handleLogin}>Sign in</button>
 </>
);
}

function Dashboard() {
 const loc = useLocation();
 return (
 <
 <h2>Dashboard</h2>
 {loc.state?.justEnrolled && <p>Welcome aboard!  </p>}
 </>
);
}

function NotFound() {
 const navigate = useNavigate();

```

```

return (
 <>
 <h2>404 Not Found</h2>
 <button onClick={() => navigate('/', { replace: true })}>Go Home</button>
 </>
);
}

export default App;

```

---

## 31. Dynamic Path

### ✓ What is a Dynamic Path?

A **dynamic path** (also called a **route parameter**) is a route segment that acts like a variable in the URL. Example: `/courses/:courseId`

- `:courseId` is **dynamic**—it matches `/courses/42`, `/courses/react-basics`, etc.

You then read that parameter inside the component to fetch data, render details, or navigate further.

---

### ✓ Why do we use Dynamic Paths?

- **Detail pages:** Course, product, user profiles (e.g., `/courses/:courseId`).
  - **Nested resources:** Course → Lesson → Comment (e.g., `/courses/:courseId/lessons/:lessonId`).
  - **SEO-friendly URLs:** Use readable slugs like `/courses/react-for-beginners`.
  - **Single UI component, many items:** One route renders many different instances based on the URL.
- 

### ✖ Core API: Define, Read, and Use Params

#### 1) Define a dynamic route

```

import { BrowserRouter, Routes, Route } from 'react-router-dom';
import CoursePage from './CoursePage';

```

```

export default function App() {
 return (
 <BrowserRouter>

```

```

 <Routes>

 <Route path="/courses/:courseId" element={<CoursePage />} />

 </Routes>

 </BrowserRouter>

);
}

```

## 2) Read the param in the component

```
import { useParams } from 'react-router-dom';
```

```

export default function CoursePage() {
 const { courseId } = useParams(); // e.g., "42" or "react-basics"
 return <h2>Course: {courseId}</h2>;
}

```

## 3) Generate links to dynamic paths

```

import { Link } from 'react-router-dom';

function CourseList({ courses }) {
 return (

 {courses.map(c => (
 <li key={c.id}>
 <Link to={` /courses/${c.id} `}>{c.title}</Link>

))}

);
}

```

---

## With Nested Dynamic Routes

Example: course detail → specific lesson

```
import { Routes, Route, Outlet, useParams } from 'react-router-dom';
```

```

function CoursesLayout() {
 return (
 <div>

```



```

 <h1>Courses</h1>

 <Outlet />

 </div>

);
}

function LessonPage() {
 const { courseId, lessonId } = useParams();
 return <p>Course {courseId} • Lesson {lessonId}</p>;
}

export default function App() {
 return (
 <BrowserRouter>
 <Routes>
 <Route path="courses" element={<CoursesLayout />}>
 <Route path=":courseId/lessons/:lessonId" element={<LessonPage />} />
 </Route>
 </Routes>
 </BrowserRouter>
);
}

```

---

 Data Router (loader) using params (recommended for data fetching)

```

import { createBrowserRouter, RouterProvider, useLoaderData } from 'react-router-dom';

async function courseLoader({ params }) {
 // params.courseId available here
 const res = await fetch(`/api/courses/${params.courseId}`);
 if (!res.ok) throw new Response('Not found', { status: 404 });
 return res.json();
}

function CoursePage() {

```

```

const course = useLoaderData(); // data from loader
return <h2>{course.title}</h2>;
}

const router = createBrowserRouter([
 { path: '/courses/:courseId', element: <CoursePage />, loader: courseLoader }
]);

export default function App() {
 return <RouterProvider router={router} />;
}

```

---

## Advanced Patterns & Tips

### 1) Optional segments and “splat” (catch-all)

- **Catch-all:** \* matches the rest of the path.

```
<Route path="/docs/*" element={<DocsRouter />} />
```

// Matches /docs, /docs/getting-started, /docs/reference/api, etc.

- React Router v6 doesn't have optional segments (like `:id?`) directly; use separate routes if needed:

```
<Route path="/courses" element={<Courses />} />
```

```
<Route path="/courses/:courseId" element={<CoursePage />} />
```

### 2) Validate/normalize params

Params are **strings**. Convert to numbers or validate slugs:

```
const { courseId } = useParams();
```

```
const id = Number(courseId);
```

```
if (Number.isNaN(id)) return <p>Invalid course id</p>;
```

### 3) Relative dynamic links (cleaner in nested routes)

// Inside /courses/:courseId route

```
<Link to="lessons/1">Lesson 1</Link> // => /courses/:courseId/lessons/1
```

```
<Link to="..">Back to courses</Link> // => /courses
```

### 4) TypeScript: type-safe params

```
import { useParams } from 'react-router-dom';
```

```
type Params = { courseId: string; lessonId?: string };
const { courseId, lessonId } = useParams<Params>();
```

### 5) Building URLs safely (helpers)

```
export const paths = {
 course: (id: string | number) => `/courses/${id}`,
 lesson: (courseId: string | number, lessonId: string | number) =>
 `/courses/${courseId}/lessons/${lessonId}`,
};
```

Use:

```
<Link to={paths.course(42)}>Open</Link>
navigate(paths.lesson(42, 1));
```

### Compact, Real-World Example (Courses → Lessons)

```
import {
 BrowserRouter, Routes, Route, Link, useParams, useNavigate
} from 'react-router-dom';

function CourseList() {
 const courses = [
 { id: 1, title: 'React Basics' },
 { id: 2, title: 'Advanced Hooks' }
];
 return (

 {courses.map(c => (
 <li key={c.id}>
 <Link to={`/${c.id}`}>{c.title}</Link>

))}

);
}

function CoursePage() {
 const { courseId } = useParams();
 const navigate = useNavigate();
 return (
 <>
 <h2>Course #{courseId}</h2>
 <button onClick={() => navigate(`lessons/1`)}>Go to Lesson 1</button>

 <Link to={`lessons/1`}>Lesson 1</Link>
 <Link to={`lessons/2`}>Lesson 2</Link>

 </>
);
}
```

```


 </>
);
}

function LessonPage() {
 const { courseId, lessonId } = useParams();
 return <p>Course {courseId} → Lesson {lessonId}</p>;
}

export default function App() {
 return (
 <BrowserRouter>
 <Routes>
 <Route path="/courses" element={<CourseList />} />
 <Route path="/courses/:courseId" element={<CoursePage />} />
 <Route path="/courses/:courseId/lessons/:lessonId" element={<LessonPage />} />
 <Route path="*" element={<p>Not Found</p>} />
 </Routes>
 </BrowserRouter>
);
}

```

### Explanation:

- `:courseId` and `:lessonId` are dynamic segments.
- `CoursePage` and `LessonPage` read params via `useParams()`.
- `Links` and `navigate()` build paths using those dynamic segments.
- You can fetch data keyed by `courseId/lessonId` and render the correct detail.

---

### Quick Recap

- **Dynamic Path** = URL segment variable (e.g., `:id`) to render different content for different resources.
- Use `useParams()` to access them.
- Prefer **data routers** (`loader`) for fetching by params.
- Validate/convert params (they're strings).

Use **relative links** in nested routes for cleaner code.

---

## 32. React Component Lifecycle

✅ Lifecycle Phases in React (Functional Components with Hooks)

React components have **three main phases**:

1. **Mounting** – Component is created and inserted into the DOM.
2. **Updating** – Component re-renders when props or state change.
3. **Unmounting** – Component is removed from the DOM.

---

✓ How Lifecycle Works in Class Components (for reference)

- **Mounting:** `constructor` → `render` → `componentDidMount`
- **Updating:** `render` → `componentDidUpdate`
- **Unmounting:** `componentWillUnmount`

---

✓ In Functional Components (with Hooks)

We use `useEffect` to handle lifecycle behaviors:

- **Mounting:** `useEffect(() => { ... }, [])` runs once after initial render.
- **Updating:** `useEffect(() => { ... }, [dependencies])` runs when dependencies change.
- **Unmounting:** Cleanup function inside `useEffect` runs before component unmounts.

---

✓ Example: Demonstrating All Phases

```
import React, { useEffect, useState } from 'react';
```

```
function LifecycleDemo() {
 const [count, setCount] = useState(0);

 // Mounting
 useEffect(() => {
 console.log('Component Mounted');
 return () => {
 console.log('Component Unmounted'); // Cleanup
 };
 }, []);

 // Updating
 useEffect(() => {
 console.log(`Count updated to ${count}`);
 }, [count]);

 return (
 <div>
 <h2>Count: {count}</h2>
 <button onClick={() => setCount(c => c + 1)}>Increment</button>
 </div>
);
}
```

export default LifecycleDemo;

---

#### ✓ Explanation of Example

- **Mounting:** First `useEffect` with empty `[]` runs once → logs "Component Mounted".
  - **Updating:** Second `useEffect` runs whenever `count` changes → logs updated value.
  - **Unmounting:** Cleanup function in first `useEffect` runs when component is removed → logs "Component Unmounted".
- 

#### ✓ Why Lifecycle Matters

- **Data fetching:** Fetch API data on mount.
  - **Subscriptions:** Start on mount, clean up on unmount.
  - **Timers:** Set on mount, clear on unmount.
  - **Performance:** Avoid unnecessary re-renders by controlling updates.
- 

#### ✓ Real-World Example: Fetch Data on Mount & Cleanup

```
function UserProfile({ userId }) {
 const [user, setUser] = useState(null);

 useEffect(() => {
 let isMounted = true;
 fetch(`/api/users/${userId}`)
 .then(res => res.json())
 .then(data => {
 if (isMounted) setUser(data);
 });

 return () => {
 isMounted = false; // Prevent state update after unmount
 };
 }, [userId]);

 return user ? <h3>{user.name}</h3> : <p>Loading...</p>;
}
```

---

#### ✓ Summary:

- Lifecycle = Mount → Update → Unmount.
  - In functional components, use `useEffect` for side effects and cleanup.
  - Always clean up subscriptions, timers, and async tasks to avoid memory leaks.
-